



CSC 202 – COMPUTER PROGRAMMING II

Department: Computer Science

Course Unit: 3 Units

Semester: Second Semester

Location: Computer Lab

Course Objectives

By the end of this course, students should be able to:

1. Transition from Academic Programming to Industry Software Engineering

- Apply foundational programming knowledge (e.g., Rust) to real-world JavaScript development.
 - Understand differences between statically typed and dynamically typed environments.
-

2. Demonstrate Structured and Modular Programming Principles

- Design and implement well-structured, maintainable, and reusable code.
 - Apply separation of concerns and layered architecture.
 - Organize code using modules, packages, and namespaces.
-

3. Apply Object-Oriented Programming Concepts in JavaScript

- Implement classes, inheritance, polymorphism, and abstraction.
 - Compare object-oriented patterns across programming languages.
 - Apply composition and design best practices in real applications.
-

4. Develop Proficiency in Modern JavaScript (ES6+)

- Use contemporary JavaScript features effectively (modules, arrow functions, destructuring, async/await, etc.).
 - Implement asynchronous and event-driven programming models.
 - Handle exceptions and errors gracefully.
-

5. Utilize TypeScript for Type-Safe Development

- Apply static typing concepts in JavaScript using TypeScript.
 - Design type-safe interfaces and reusable components.
 - Appreciate the importance of type systems in large-scale software development.
-

6. Design and Implement Backend Web Services

- Develop RESTful APIs using Node.js.
 - Structure backend applications using controllers, middleware, and services.
 - Integrate applications with databases for persistent data storage.
-

7. Build Interactive Frontend Applications

- Develop component-based user interfaces using React.
 - Implement state management and API integration.
 - Apply reusable component design principles.
-

8. Understand Cross-Platform Development Concepts

- Explain the principles behind building for Web, Mobile, and Desktop environments.
 - Demonstrate awareness of React Native and shared logic patterns.
-

9. Apply Basic DevOps and Version Control Practices

- Use Git for collaborative development.
- Implement basic CI/CD workflows.
- Deploy applications to modern cloud platforms.

10. Develop Professional Software Development Competence

- Write clean, readable, and maintainable code.
- Document projects effectively.
- Prepare portfolio-ready, deployable applications suitable for internships.

Learning Outcomes

- Explain Core Programming Concepts in JavaScript
- Apply Structured and Modular Programming Principles
- Apply Structured and Modular Programming Principles
- Implement Object-Oriented Programming Concepts
- Develop Type-Safe Applications Using TypeScript
- Design and Implement RESTful Backend Services
- Build Interactive Web Applications Using React
- Apply Basic DevOps Practices
- Demonstrate Professional Software Development Skills

Method of Delivery

- Lectures (concepts and principles)
- Hands-on Programming
- Projects and Assignments
- Quizzes and Tests

Method of Assessment

Assessment	Weighting	Details
Continuous Assessment	30%	Quizzes, Test Project Presentation
Class Participation	5%	Attendance & Active Involvement
End of Semester Exam	65%	Formal Examination

Weekly Breakdown

Week	TOPICS	Key Deliverables
Week 1: Course Orientation & Modern Programming Mindset -	Intro to VS Code, Git, and Node.js. Transitioning from basic scripts to Structured Programming.	Hello World in Node.js & Git Repo.

Modern Development Ecosystem		
Week 2: JavaScript Fundamentals (ES6+)	Modern syntax: Arrow functions, Destructuring, Spread, and Modules	Modular CLI Calculator.
Week 3: Object-Oriented Programming in JS	Classes & Objects: Constructors, this keyword, and Encapsulation Inheritance, Polymorphism, and Abstraction:	Building a Payment Processor hierarchy.
Week 4: TypeScript & Type Safety	Interfaces, Types, and Enums. Bridging OOP concepts with static typing.	Convert JS project to TS
Week 5: Backend Basics	Understanding APIs, Routing, and Middleware as "Event-driven programming"	A simple REST API. : Build API for main semester project
Week 6: COMPUTER BASE OBJECTIVE TEST		
Week 7: Database Integration	Intro to Databases (SQL/NoSQL) and CI/CD basics (GitHub Actions).	Connecting API to a DB.
Week 8: Frontend: React.js (The Web)	Components, State, and Props. Building "Solutions for the Web".	Interactive Web Dashboard.
Week 9: Advanced React + Architecture	Folder structure Reusable components Context API Error boundaries Form handling	Create a polished dashboard
Week 10: WRITTEN ASSESSMENT		
Week 11: Mobile: React Native (Cross-Platform)	Shared logic between Web and Mobile. Intro to "Native codes with React-Native"	First Mobile App Screen.
Week 12: Blockchain & Future Tech	Conceptual overview: Web3, Smart Contracts (Solidity), and Metaverse.	Web3 "Connect Wallet" button.
Week 13: Intro to DevOps	Git branching	Deploy project live

	Pull requests CI with GitHub Actions Deployment (Vercel + Render) Docker (intro only)	
Week 14: Project Presentations & Revision	Project presentations	Revision